



FOR JAVA DEVELOPERS UPGRADING SPRING BOOT

Upgrade passed. Orders failed.

Spring Boot 4: what actually changed, 3.5 to 4.1, on one real service

Loud breaks stop the build. Silent ones pass it. Know which gate catches each.

One Spring Boot 3.5 orders service, upgraded to 4.1 step by step, following Spring's own migration guide. Every change is sorted into three kinds. Loud: the compiler, the tests or startup stop you. Silent: compile, unit tests and startup pass, but behaviour the app relied on is gone. Opt-in: nothing breaks, and Boot 4 gives you something new.

Silent does not mean undocumented. Spring documents every silent change here. It means the compile, unit test and startup gates did not catch it.

At the time of writing, October 2026: Spring Boot 4.1.1 is current, 4.0.0 shipped on 20 November 2025, and 3.5.16 was the last open source release of the 3.5 line. Spring Boot 4.2 is planned for November 2026. Newer versions may differ.

This app exercised

- Starters and modules: Undertow, Flyway, the REST client, the test slices
- Jackson 3, and a JSON contract that changed on the wire
- The properties migrator, and a key it reported but did not repair
- API versioning, HTTP service clients and NullAway, as opt-ins

Not covered

- Spring Security 7, and Hibernate 7 beyond what this app touches
- gRPC support, new in 4.1; @Retryable and @ConcurrencyLimit
- Virtual threads; liveness and readiness probes now on by default
- Batch, Kafka, AMQP, MongoDB, Redis and Session property moves

Every branch of the repository is one step of the upgrade, and scripts/verify.sh replays them all on your machine.

01 The migration ledger

No Undertow starter in Boot 4	Loud: Maven could not read the POM
Testcontainers 2 module names (not Spring)	Loud: Maven could not read the POM
Jackson 3: tools.jackson, @JacksonComponent	Loud: the compiler
RestTemplateBuilder in the REST client module	Loud: the compiler
Test slices and TestRestTemplate moved	Loud: the compiler, on the tests
@MockBean removed	Loud if step 1 is skipped; 3.5 already warned
flyway-core alone runs no migrations	Silent: only the Testcontainers tests
Our ObjectMapper bean no longer shapes HTTP JSON	Silent: only the tests that assert the JSON
server.error.include-message	Reported by the migrator, not repaired
API versioning, HTTP service clients, NullAway	Opt-in

Kind, then the gate that caught it. Both silent changes were caught only by a test that looked at the real thing: a real database, the real JSON.

02 Build green. Schema missing.

With only flyway-core, Spring Boot 3.5 configured Flyway because the library was on the class path. On 4.1 the startup log has no Flyway line at all, ./mvnw package is green with 29 tests, the app starts, and the first POST /orders returns 500.

Spring Boot 4 is split into modules, one per technology. For technologies like Flyway that used to switch on from the library alone, Boot 4 expects the module, through its starter.

```
● ● ● pom.xml, step-4-flyway-starter (git diff)
```

```
<dependency>
-   <groupId>org.flywaydb</groupId>
-   <artifactId>flyway-core</artifactId>
+   <groupId>org.springframework.boot</groupId>
+   <artifactId>spring-boot-starter-flyway</artifactId>
</dependency>
```

```
● ● ● Before the starter, then after it (evidence/step-3 and step-4)
```

```
org.postgresql.util.PSQLException: ERROR: relation "orders" does not exist
```

```
o.f.core.internal.command.DbMigrate      : Successfully applied 1 migration to schema "public",
now at version v1 (execution time 00:00.008s)
```

```
● ● ● ./mvnw verify on the same commit, without the starter (evidence/step-3/build-verify.txt)
```

```
[ERROR] Tests run: 10, Failures: 3, Errors: 5, Skipped: 0
[INFO] BUILD FAILURE
```

The only gate that caught it ran against a real PostgreSQL in Testcontainers. Unit tests with a mocked repository never touch the schema.

flyway-core alone: no Flyway line at startup, 29 unit and slice tests green, POST /orders 500. With spring-boot-starter-flyway: "Successfully applied 1 migration", POST /orders 201, 39 tests green.

Measured on a real run, 2026-10-07: evidence/ in the repository, reproduced by scripts/verify.sh.

The spring-boot jars in BOOT-INF/lib went from 7 on 3.5 to 26 on 4.1; spring-boot-autoconfigure from 2,088,460 to 372,740 bytes.

Measured on a real run, 2026-10-07: evidence/ in the repository, reproduced by scripts/verify.sh.

Flyway needs spring-boot-starter-flyway. spring-boot-starter-classic and spring-boot-starter-test-classic exist as a temporary bridge.

Modularizing Spring Boot, spring.io/blog/2025/10/28/modularizing-spring-boot, checked 2026-10-07.

03 Same bean. Different JSON.

The contract: snake_case field names and UK dates, dd/MM/yyyy. On 3.5 our own ObjectMapper bean set those rules. On 4.1, ported to Jackson 3 and still declared as an ObjectMapper, it compiles and starts, but it no longer replaces the JsonMapper Boot auto-configures, so the rules are not applied.

The better migration for this app is to customise the mapper Boot already builds, so @JacksonComponent serializers and spring.jackson.* properties still apply.

Spring Boot 3.5: JacksonConfig.java, step-0-boot-3.5 (excerpt)

```
@Bean
public ObjectMapper objectMapper(
    Jackson2ObjectMapperBuilder builder) {
    var writer = new LocalDateSerializer(UK_DATE);
    var reader = new LocalDateDeserializer(UK_DATE);
    return builder
        .propertyNamingStrategy(SNAKE_CASE)
        .featuresToDisable(WRITE_DATES_AS_TIMESTAMPS)
        .serializerByType(LocalDate.class, writer)
        .deserializerByType(LocalDate.class, reader)
        .build();
}
```



Spring Boot 4.1: JacksonConfig.java, main (excerpt)

```
import static tools.jackson.databind.PropertyNamingStrategies.SNAKE_CASE;
import static tools.jackson.databind.cfg.DateTimeFeature.WRITE_DATES_AS_TIMESTAMPS;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;

import tools.jackson.databind.ext.javatime.deser.LocalDateDeserializer;
import tools.jackson.databind.ext.javatime.ser.LocalDateSerializer;
import tools.jackson.databind.module.SimpleModule;

import org.springframework.boot.jackson.autoconfigure.JsonMapperBuilderCustomizer;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class JacksonConfig {

    public static final DateTimeFormatter UK_DATE =
        DateTimeFormatter.ofPattern("dd/MM/yyyy");

    @Bean
    public JsonMapperBuilderCustomizer ordersJsonContract() {
        return builder -> builder
            .propertyNamingStrategy(SNAKE_CASE)
            .disable(WRITE_DATES_AS_TIMESTAMPS)
            .addModule(ukDates());
    }

    private static SimpleModule ukDates() {
        return new SimpleModule("uk-dates")
            .addSerializer(LocalDate.class,
                new LocalDateSerializer(UK_DATE))
            .addDeserializer(LocalDate.class,
                new LocalDateDeserializer(UK_DATE));
    }
}
```

GET /orders/1, byte for byte (evidence/step-5/json-compare.txt, excerpt)

```
3.5:          "customer_name":"Sarah Thompson" ... "placed_on":"07/10/2026"
4.1, our bean: "customerName":"Sarah Thompson" ... "placedOn":"2026-10-07"
```

3.5 and 4.1 with the customizer: IDENTICAL

3.5 and 4.1 with the ObjectMapper bean: DIFFERENT

The first two lines are shortened with "..."; the full bodies are in the transcript. With the ObjectMapper bean, a snake_case POST /orders got HTTP 400.



●●● The tests that caught it: @WebMvcTest, asserting the JSON (evidence/step-3)

```
[ERROR] OrderControllerWebMvcTest.createAnOrder:52 Status expected:<201> but was:<400>
[ERROR] OrderControllerWebMvcTest.writesTheAgreedJsonContract:63 No value at JSON path
"$$.customer_name"
```

A user ObjectMapper bean no longer shapes the HTTP JSON in Boot 4; define a JsonMapper, or customise Boot's.

Spring Boot 4.0 Migration Guide, github.com/spring-projects/spring-boot/wiki, checked 2026-10-07.

With the JsonMapperBuilderCustomizer, GET /orders/1 is byte for byte identical to 3.5.

Measured on a real run, 2026-10-07: evidence/ in the repository, reproduced by scripts/verify.sh.

04 Testing is modular now too

@MockBean is gone in 4.0. The 3.5.16 compiler already reports it "deprecated and marked for removal", so replace it on 3.5, where the fix is easy to verify. The test slices moved into their own modules and test starters, and a full @SpringBootTest asks for TestRestTemplate with its own annotation.

OrderControllerWebMvcTest.java, step-1-latest-3.5 (git diff, excerpt)

```
-import org.springframework.boot.test.mock.mockito.MockBean;
+import org.springframework.test.context.bean.override.mockito.MockitoBean;

- @MockBean
+ @MockitoBean
    private OrderService service;
```

pom.xml, step-3c-test-starters (git diff)

```
<dependency>
  <groupId>org.springframework.boot</groupId>
-  <artifactId>spring-boot-starter-test</artifactId>
+  <artifactId>spring-boot-starter-webmvc-test</artifactId>
+  <scope>test</scope>
+</dependency>
+<dependency>
+  <groupId>org.springframework.boot</groupId>
+  <artifactId>spring-boot-starter-restclient-test</artifactId>
+  <scope>test</scope>
+</dependency>
```

The moved imports, collected from four test files, step-3c-test-starters

```
import org.springframework.boot.webmvc.test.autoconfigure.WebMvcTest;
import org.springframework.boot.webmvc.test.autoconfigure.AutoConfigureMockMvc;
import org.springframework.boot.restclient.test.autoconfigure.RestClientTest;
import org.springframework.boot.resttestclient.TestRestTemplate;
import org.springframework.boot.resttestclient.autoconfigure.AutoConfigureTestRestTemplate;
```

OrdersRestTestClientIT.java, main (excerpt): the new fluent option

```
@AutoConfigureRestTestClient
@Import(TestcontainersConfiguration.class)
@ActiveProfiles("local-stub")
class OrdersRestTestClientIT {

    @Autowired
    private RestTestClient client;

    @Test
    void placesAnOrderEndToEnd() {
        client.post().uri("/orders")
            .contentType(APPLICATION_JSON)
            .body(TestOrders.SARAH)
            .exchange()
            .expectStatus().isCreated()
            .expectBody()
            .jsonPath("$.payment_status")
            .isEqualTo("AUTHORISED")
            .jsonPath("$.total")
            .isEqualTo("36.50 GBP");
    }
}
```

The untouched 3.5 tests compiled against 4.1: "package org.springframework.boot.test.mock.mockito does not exist", "symbol: class MockBean".

Measured on a real run, 2026-10-07: evidence/ in the repository, reproduced by scripts/verify.sh.

Test slices moved to per technology test modules; TestRestTemplate needs
@AutoConfigureTestRestTemplate.

Spring Boot 4.0 Migration Guide, github.com/spring-projects/spring-boot/wiki, checked 2026-10-07.

05 Opt-in: API versioning in one attribute

The version is an attribute on the mapping, and two properties say where it comes from and what the default is. A deprecated version takes a small handler bean, not a property, and requests with no version resolve to the default, so the deprecation headers land on them too.

OrderController.java, main (excerpt)

```
@GetMapping(path =("/{id}", version = "1")
public OrderView find(@PathVariable long id) {
    return service.find(id);
}

@GetMapping(path =("/{id}", version = "2")
public OrderSummaryV2 findV2(@PathVariable long id) {
    return OrderSummaryV2.of(service.find(id));
}
```

application.properties, main (excerpt)

```
spring.mvc.apiversion.use.header=X-Version
spring.mvc.apiversion.default=1
```

ApiVersionConfig.java, main (excerpt)

```
@Bean
public ApiVersionDeprecationHandler deprecationHandler() {
    StandardApiVersionDeprecationHandler handler =
        new StandardApiVersionDeprecationHandler();
    handler.configureVersion("1")
        .setDeprecationDate(DEPRECATED_ON)
        .setSunsetDate(SUNSET_ON)
        .setDeprecationLink(MIGRATION_GUIDE);
    return handler;
}
```

curl results (evidence/step-6/curl.txt, status lines and bodies)

```
no header      HTTP/1.1 200   the version 1 body
X-Version: 2    HTTP/1.1 200   {"id":1,"customer_name":"Sarah Thompson","item_count":3,...}
X-Version: 3    HTTP/1.1 400   "message":"Invalid API version: '3.0.0'."
X-Version: abc  HTTP/1.1 400   "message":"Invalid API version: 'abc'."

Deprecation: @1790812800
Sunset: Thu, 1 Apr 2027 00:00:00 GMT
```



Version 3 and abc both returned 400; the deprecation headers also appeared on POST /orders, which has no version attribute.

Measured on a real run, 2026-10-07: evidence/ in the repository, reproduced by scripts/verify.sh.

Versions are mapped with the version attribute; deprecation is signalled with the Deprecation and Sunset headers.

Spring Framework reference, API versioning, checked 2026-10-07.

06 Opt-in: HTTP service clients

The payments call was a `RestTemplate` wrapped by hand. Now it is an interface: one method, the path, the body. Spring creates the client, and the base URL is one property. `RestTemplate` still works in 4.1; this is less code for the same call, not a forced move.

Before: `HttpPaymentGateway.java`, `step-5-jackson` (excerpt)

```
public HttpPaymentGateway(
    RestTemplateBuilder builder,
    @Value("${payments.base-url}") String baseUrl) {
    this.restTemplate = builder
        .baseUrl(baseUrl)
        .connectTimeout(Duration.ofSeconds(2))
        .readTimeout(Duration.ofSeconds(5))
        .build();
}

@Override
public PaymentStatus authorise(long orderId, Money amount) {
    AuthoriseRequest request = new AuthoriseRequest(
        orderId, amount.amount(), amount.currency());
    AuthorisationResponse response = restTemplate
        .postForObject(PATH, request,
            AuthorisationResponse.class);
}
```

After: `PaymentsApi.java` and `PaymentsClientConfig.java`, main

```
@HttpExchange("/payments")
interface PaymentsApi {

    @PostExchange("/authorisations")
    ResponseEntity<AuthorisationResponse> authorise(
        @RequestBody AuthoriseRequest request);
}

@Configuration
@ImportHttpServices(
    group = "payments",
    types = PaymentsApi.class)
public class PaymentsClientConfig {
}
```

application.properties, main (excerpt)

```
spring.http.serviceclient.payments.base-url=\
http://localhost:${server.port:8080}
```

Through the `@HttpExchange` client, `POST /orders` returned 201 AUTHORISED and all integration tests passed.

Measured on a real run, 2026-10-07: `evidence/` in the repository, reproduced by `scripts/verify.sh`.

`@ImportHttpServices` registers the interface as a bean; `spring.http.serviceclient.<group>.base-url` sets its base URL.

Spring Boot reference, HTTP service clients, checked 2026-10-07.

07 Opt-in: null safety the build enforces

Spring Framework 7 marks which of its methods can return null, with JSpecify annotations. Mark your package `@Nullable`, run NullAway in the build, and an unchecked nullable return stops the compile. NullAway's JSpecify mode needs JDK 21.0.8 or newer, or any JDK 22+.

package-info.java, main (excerpt)

```
@Nullable
package com.example.orders.payments;

import org.jspecify.annotations.Nullable;
```

pom.xml, main (excerpt): the NullAway arguments

```
<nullaway.checks>
  -XepDisableAllChecks -Xep:NullAway:ERROR
</nullaway.checks>
<nullaway.options>
  -XepOpt:NullAway:OnlyNullable=true
  -XepOpt:NullAway:JSpecifyMode=true
</nullaway.options>
```

The compiler plugin execution that passes them to Error Prone is in the same pom.xml.

HttpPaymentGateway.java, step-6c-nullaway-fails (excerpt, lines 26 to 28)

```
AuthorisationResponse response =
    api.authorise(request).getBody();
return PaymentStatus.valueOf(response.status());
```

[NullAway] dereferenced expression 'response' is @Nullable

The fix, main (excerpt)

```
AuthorisationResponse response =
    api.authorise(request).getBody();
if (response == null) {
    String message = "Payments returned an empty body"
        + " for order " + orderId;
    throw new IllegalStateException(message);
}
return PaymentStatus.valueOf(response.status());
```



The unchecked `getBody()` failed the compile with the NullAway error above; the checked version builds and all tests pass.

Measured on a real run, 2026-10-07: `evidence/` in the repository, reproduced by `scripts/verify.sh`.

Spring Framework 7 publishes its nullness with JSpecify annotations; NullAway is the recommended checker.

Spring Framework reference, null safety, checked 2026-10-07.

The upgrade, in the order that works

1. Get onto the latest 3.5
2. Clear the Spring Boot 3 deprecations, while the fix is easy to verify
3. Run the properties migrator, and read its report: it may report a key without repairing it
4. Check every technology starter: a library on the class path is no longer enough for technologies like Flyway
5. Check the test starters, and ask for `TestRestTemplate` and `MockMvc` explicitly
6. Test your database migrations against a real database
7. Assert the JSON, not just the objects
8. Then remove the bridges: the properties migrator, and the classic starters if you used them



Everything else

-  **Site**
<https://code-with-sam-dev.github.io>
-  **YouTube**
<https://www.youtube.com/@CodewithSam-Dev>
-  **LinkedIn**
<https://www.linkedin.com/company/code-with-sam-dev>
-  **x**
<https://x.com/CodeWithSamDev>
-  **TikTok**
<https://www.tiktok.com/@codewithsamdev>
-  **Instagram**
<https://www.instagram.com/codewithsamdev>
-  **Facebook**
<https://www.facebook.com/codewithsamdev>
-  **GitHub**
<https://github.com/code-with-sam-dev>
-  **Buy me a coffee**
<https://buymeacoffee.com/codewithsam>
-  **Buy me sadza**
<https://buymesadza.com/codewithsam>
- The repository: every step a branch, plus scripts/verify.sh**
<https://github.com/code-with-sam-dev/spring-boot-4-upgrade>
- Spring Boot 4.0 Migration Guide**
<https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-4.0-Migration-Guide>
- Modularizing Spring Boot**
<https://spring.io/blog/2025/10/28/modularizing-spring-boot/>
- API versioning**
<https://docs.spring.io/spring-framework/reference/web/webmvc-versioning.html>
- HTTP service clients**
<https://docs.spring.io/spring-boot/reference/io/rest-client.html>
- Null safety**
<https://docs.spring.io/spring-framework/reference/core/null-safety.html>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which upgrade or system you want taken apart next.

Java is a trademark of Oracle. Spring and Spring Boot are trademarks of Broadcom. PostgreSQL is a trademark of the PostgreSQL Community Association. Jackson, Flyway, Testcontainers and NullAway are the property of their respective owners. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-10-07 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.